

Min-Cut Algorithm Lecture



L10_S1.pdf

PDF

Task: Generate a lecture scribe.

Purpose: A lecture scribe is a faithful, exam-ready reconstruction of what was taught in class. The scribe should function as reference material suitable for preparing for a reading-based / closed-notes exam.

The context is fixed and authoritative.

Use ONLY the provided lecture slides. The lecture scribe guidelines govern how the scribe must be written.

Everything included must be traceable to the provided context.

Do not use external knowledge or introduce material not present in the context.

Prompting Style:

PROMPTING STYLE 3: CHAIN-OF-THOUGHT (CoT)

PROMPTING should be followed.

Chain-of-Thought Requirements: While writing the lecture scribe, reason step by step. Reasoning may appear in the final scribe to improve clarity for exam preparation. All reasoning must come only from the provided context.

Content Requirements:

The lecture scribe should allow a student to: Recall definitions and notation, Reconstruct proofs and derivations, Follow worked examples step by step, Understand logical dependencies between concepts

Accordingly, include Definitions and notation, Assumptions and conditions, Statements of theorems or results, Proofs or proof sketches, step by step, exactly as presented, Worked examples with

intermediate steps, exactly as shown in the lecture

For algorithmic content in this lecture:

- Present pseudocode exactly as given in the slides.
- Preserve algorithm names, structure, and control flow.
- Clearly explain each step of the algorithm only if such explanation appears in the slides.
- State time complexity expressions exactly as written.
- Present theorem statements and probability guarantees precisely as stated in the lecture.
- Do not add interpretations, optimizations, or alternative algorithm descriptions.

Do NOT include:

- Creative explanations
- Simplified tutorials
- New intuition
- New examples
- Material not discussed in the lecture or shown in the slides

Explanation Constraints are- Explain definitions and assumptions explicitly.

Require step-by-step proofs, derivations, and example solutions when they appear in the lecture.

Do not provide alternative reasoning paths or additional explanations.

Structural Requirements are- Follow the structure and flow of the lecture as presented in the slides.

Organize the scribe clearly with appropriate sections and subsections so that it can be used reliably for revision.

Tone: Maintain a formal, academic tone suitable for exam revision. Prioritize correctness and faithfulness to the lecture. Do not invent content.

Finally:

Write the lecture scribe such that a student studying

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel, PhD

Date: February 5, 2026

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The lecture states that the min-cut algorithm is used in applications related to:

- **Network connectivity**
- **Reliability**
- **Optimization**

Specifically:

1. Network Design

- Min-cut helps in improving the efficiency of communication.
- It is used to optimize network flow.
- The algorithm is used to find the **minimum capacity cut**.

2. Communication Networks

- It helps in understanding the vulnerability of networks to failures.
- It supports building robust and fault-tolerant communication networks.

3. VLSI Design

- In Very Large Scale Integration (VLSI) design, min-cut is useful for partitioning circuits into smaller components.
- This reduces interconnectivity complexity.

(Reference: Section 1.5, *Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition)*, as mentioned in the slide .)

1.2 What is Min-Cut?

Definition: Cut-Set

A **cut-set** in a graph is:

A set of edges whose removal breaks the graph into two or more connected components.

Definition: Min-Cut Problem

Given a graph

$$G = (V, E)$$

with n vertices,

The **minimum cut (min-cut) problem** is:

To find a minimum cardinality cut-set in G .

Thus:

- Input: Graph $G = (V, E)$
 - Output: A cut-set with minimum number of edges
-

1.3 Randomness in Min-Cut Algorithms

The slides state:

- Min-cut algorithms such as **Karger's algorithm** are random.
- They can be sensitive to the initial choice of edges.
- If the algorithm contracts critical edges early, it might find a smaller cut.

Thus, correctness depends on contraction choices.

1.4 Edge Contraction (Main Operation)

The main operation in the min-cut algorithm is:

Edge contraction

Definition: Edge Contraction

Edge contraction is:

An operation that removes an edge from a graph while simultaneously merging

the two vertices.

Detailed Description (from slide)

When contracting an edge (u, v) :

1. Merge vertices u and v into a single vertex.
2. Eliminate all edges connecting u and v .
3. Retain all other edges in the graph.
4. The new graph:
 - May have parallel edges.
 - Has no self-loops.

This operation is illustrated in the diagram on pages 15–16 of the slides .

2. Successful and Unsuccessful Min-Cut Runs

2.1 Successful Min-Cut Run

Definition:

A successful min-cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

The figure labeled “Successful Run” illustrates a sequence of contractions that preserve the minimum cut until the final two vertices remain .

Thus:

- The algorithm contracts edges.
 - The true minimum cut is not destroyed.
 - The final cut corresponds to the minimum cut.
-

2.2 Unsuccessful Min-Cut Run

Definition:

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The slide explicitly states:

- The algorithm fails to correctly identify the minimum cut.

The figure labeled “Unsuccessful Run” shows contraction of edges that eliminate the true minimum cut .

Thus:

- A critical edge is contracted.
- The minimum cut is destroyed.
- The final result is not the true minimum cut.

3. Max-Flow Min-Cut Theorem

The theorem states:

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions from Slide

1. Capacity of a cut

= The sum of the capacity of edges in the cut that are oriented from a vertex $\in X$ to a vertex $\in Y$.

2. Minimum cut

= The cut in the network that has the smallest possible capacity.

3. Minimum cut capacity

= The capacity of the minimum cut.

4. Maximum flow

= The largest possible flow from source S to sink T .

The diagram on page 26 illustrates a flow network with source S , sink T , and intermediate vertices .

Thus:

$$\text{Maximum Flow} = \text{Minimum Cut Capacity}$$

4. Deterministic Min-Cut Algorithm

4.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of graph G .

Let $G/\{s, t\}$ be the graph obtained by merging s and t .

The theorem states:

A minimum cut of G can be obtained by taking the smaller of:

- "A minimum s - t -cut of G , and"
- "A minimum cut of $G/\{s, t\}$."

Reason (as given in slide):

Either:

1. There exists a minimum cut that separates s and t .
→ Then a minimum s - t -cut of G is a minimum cut of G .

OR

2. There is no such cut separating s and t .
→ Then a minimum cut of $G/\{s, t\}$ gives the answer.

4.2 Pseudocode (As Given)

Algorithm 1: MinimumCutPhase(G, a)

```
pgsql
A ← {a};
while A ≠ V do
    add to A the most tightly connected vertex;
return the cut weight as the "cut of the phase";
```

Algorithm 2: MinimumCut(G)

```
pgsql
while |V| ≥ 1 do
    choose any a from V;
    MinimumCutPhase( $G, a$ );
    if the cut-of-the-phase is lighter than the current minimum cut then
```

```
store the cut-of-the-phase as the current minimum cut;  
shrink G by merging the two vertices added last;  
return the minimum cut;
```

5. Randomized Min-Cut Algorithm

5.1 Why Randomized Algorithm?

The slides state:

Randomized algorithms:

- Provide a probabilistic guarantee of success.
- Provide a more accurate estimate of the minimum cut with fewer iterations.

Additional properties listed:

- Efficiency
 - Parallelization
 - Approximation Guarantees
 - Avoidance of Worst-Case Instances
 - Heuristic Nature
 - Robustness
-

5.2 Karger's Randomized Algorithm

The diagram on page 34 shows:

- Random selection of an edge.
- Edge contraction.
- Repetition until two vertices remain.
- The remaining parallel edges form the cut.

The example shows:

- A run where random edges are picked.
- The output cut is {a, c, e}.
- But the minimal cut is either {b, e} or {a, d}.

Thus:

- Random selection may or may not preserve the minimum cut.

5.3 Pseudocode (Exact from Slide)

Algorithm 3: Recursive-Randomized-Min-Cut(G, α)

Input:

- An undirected multigraph G with n vertices
- An integer constant $\alpha > 0$

Output:

- A cut C of G

vbnet

```
if  $n \leq \alpha^3$  then
     $C \leftarrow$  a min-cut of  $G$  found using brute force (exhaustive) search;
else
    for  $i \leftarrow 1$  to  $\alpha$  do
         $G' \leftarrow$  multigraph obtained by applying  $n - \lceil n/\sqrt{\alpha} \rceil$  random contraction steps on  $G$ ;
         $C' \leftarrow$  Recursive-Randomized-Min-Cut( $G', \alpha$ );
        if  $i = 1$  or  $|C'| < |C|$  then
             $C \leftarrow C'$ ;
return  $C$ ;
```

6. Comparison: Deterministic vs Randomized Min-Cut

The slides state:

Any specific problem is responsible for deciding the approach.

Deterministic Min-Cut

- Always guarantees an exact minimum cut.
- May have higher time complexity for large graphs.
- Stoer-Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

Where:

- V = number of vertices

- E = number of edges
-

Randomized Min-Cut

- Provides an approximate minimum cut with high probability.
- Karger's algorithm time complexity:

$$O(V^2)$$

Where:

- V = number of vertices
-

7. Theorem for Min-Cut Set

The slide states:

The algorithm outputs a min-cut set with probability at least $\frac{2}{n(n-1)}$.

Thus:

- Success probability $\geq \frac{2}{n(n-1)}$
-

8. Python Simulation

The slide instructs:

- Open Campuswire post regarding today's lecture (L10)
 - Download the `.ipynb` file pasted in the comments
-

Conclusion

This lecture covered:

1. Definition and applications of min-cut.
2. Edge contraction as the main operation.
3. Successful vs unsuccessful runs.
4. Max-Flow Min-Cut theorem.
5. Stoer-Wagner deterministic algorithm and pseudocode.

6. Karger's randomized algorithm and recursive pseudocode.
7. Comparison of deterministic and randomized approaches.
8. Probability guarantee theorem for min-cut.

This completes the faithful reconstruction of Lecture 10 based strictly on the provided slides .